

AL2002 - Artificial Intelligence Lab

Project Lab Manual

A Multi-Domain AI Learning Framework

National University of Computer and Emerging Sciences
Chiniot - Faisalabad Campus

Section 1: Introduction

1.1 Purpose of This Manual

This lab manual is designed for students enrolled in AL2002 - Artificial Intelligence Lab. It provides a structured, step-by-step guide for completing a full-semester project that covers all the core concepts taught in the course. The manual is not just a set of coding exercises. It is a research-oriented framework that encourages students to think critically, compare methods, and draw conclusions from real data.

1.2 What Students Will Build

Each group (3 members) will build an AI system that applies all major AI techniques to a real-world dataset. The system will go through multiple phases, starting from basic Python programming and ending with neural network implementation. At every phase, students will document their observations, compare results, and analyze why certain techniques work better than others.

Section 2: Dataset Tracks

2.1 Overview

Every student is assigned one Track at the start of the semester. All Tracks cover the same 15 experiments. The difference is only in the dataset used. This ensures academic integrity, as no two students work on the same data.

All datasets are freely available on Kaggle. Students must download their assigned dataset before Phase 1 begins.

Track	Domain	Dataset Name	Kaggle Search Keyword
Track A	Medical Diagnosis	Disease Symptom Prediction Dataset	disease symptom prediction itachi9604
Track B	Agriculture	Crop Disease Dataset	crop disease classification
Track C	Education	Student Performance Dataset	student performance prediction UCI
Track D	Mental Health	Mental Health in Tech Survey	mental health tech survey OSMI

2.2 What Each Track Contains

Track A - Medical Diagnosis

This dataset contains 41 diseases and their associated symptoms. Each row maps a set of symptoms to a disease label. Additional files provide symptom severity scores and precautions. This makes it suitable for weighted search algorithms and neural network classification.

Track B - Agriculture

This dataset contains records of crop conditions, environmental factors such as soil type, temperature, and rainfall, and corresponding disease or yield outcomes. It is useful for search-based optimization and supervised classification.

Track C - Education

This dataset contains student demographic data, study habits, family background, and final grades. It is suitable for clustering students into performance groups and for predicting outcomes using machine learning.

Track D - Mental Health

This dataset is a survey of technology workers containing responses about mental health conditions, workplace support, and treatment history. It is suitable for logic-based reasoning and binary classification using neural networks.

Section 3: Project Structure

The project is divided into six phases. Each phase corresponds to a group of experiments from the course outline. Students must complete each phase before moving to the next. Each phase ends with a brief written reflection that will contribute to the final research report.

Phase	Title	Experiments Covered	CLO
Phase 1	Python and Data Foundation	Experiments 1 and 2	CLO 1, 5
Phase 2	Intelligent Agent and Search	Experiments 3 to 8	CLO 1, 5
Phase 3	Constraint Satisfaction	Experiments 9 and 10	CLO 2, 5
Phase 4	Logic and Reasoning	Experiments 11 and 12	CLO 3, 5
Phase 5	Machine Learning	Experiments 13 to 15	CLO 4, 5
Phase 6	Research Report and Evaluation	Project Evaluation (Experiment 16)	CLO 1 to 5

Phase 1: Python and Data Foundation

Objective

In this phase, students will set up their Python programming environment, load their assigned dataset, and perform an initial exploration. This phase lays the foundation for all subsequent work. Without a clean, well-understood dataset, later experiments will produce unreliable results.

Step 1: Set Up Your Environment

Before writing any code, you must prepare your working environment. You may use any of the following tools. Jupyter Notebook or Google Colab are recommended for beginners because they allow you to run code in small blocks and see results immediately.

1. Python Interpreter: The base Python installation.
2. Jupyter Notebook: Runs code in cells, ideal for data exploration.

Install the required libraries before proceeding. Run the following command:

```
pip install pandas numpy matplotlib scikit-learn
```

Step 2: Download Your Dataset

Go to [kaggle.com](https://www.kaggle.com) and create a free account. Search for your Track dataset using the keyword provided in Section 2. Download the CSV files to your working directory. If you are using Google Colab, upload the files using the file upload panel on the left sidebar.

Once downloaded, place all CSV files in a folder named after your track. For example, if you are on Track A, name the folder `track_a_medical`.

Step 3: Load and Inspect the Dataset

Write a Python script to load your dataset. Use the pandas library to read the CSV file. Once loaded, perform the following inspection steps in order.

1. Print the first 10 rows of the dataset using the `head()` function.
2. Print the shape of the dataset, meaning the number of rows and columns.
3. Print the data types of each column using the `dtypes` attribute.
4. Check for missing values using the `isnull().sum()` function.

5. Print the number of unique values in the target column, which is the column you want to predict.

Record your findings in the Phase 1 Reflection section at the end of this phase. You should note: how many rows and columns the dataset has, whether there are any missing values, and what the target variable looks like.

Step 4: Apply Python Fundamentals

Using what you know about Python data types, flow controls, and data structures, complete the following tasks on your dataset.

6. Use a for loop to count how many rows belong to each class or category in the target column.
7. Store the result in a Python dictionary where the key is the class name and the value is the count.
8. Use a list to store the names of all columns in the dataset.
9. Use slicing to extract the first 50 rows and store them in a new variable called `sample_data`.
10. Write a function called `describe_dataset()` that prints the shape, column names, and class distribution when called.

Step 5: Object-Oriented Representation

Now represent your dataset using an object-oriented approach. This is important because later phases will use classes to represent agents, nodes in a graph, and logic expressions.

Create a Python class called `DataRecord`. This class should have the following properties:

3. `record_id`: A unique identifier for each row.
4. `features`: A dictionary storing the input variables for that row.
5. `label`: The target value for that row.

Write a method called `display()` inside the class that prints all properties in a readable format. Then create at least five `DataRecord` objects using rows from your dataset and call the `display()` method on each one.

Step 6: Build a Graph from Your Data

AI search algorithms operate on graphs. In this step, you will represent part of your dataset as a graph. A graph consists of nodes and edges. Each node represents a unique value in one of your key columns. Each edge connects two nodes if they appear together in the same row.

For example, in Track A, each symptom is a node. Two symptoms are connected by an edge if they both appear in the same disease entry. In Track C, each study habit category is a node connected to the grade category it most commonly produces.

11. Identify two columns in your dataset that have a meaningful relationship.
12. Extract the unique values from each column and treat them as nodes.
13. Create a Python dictionary where each key is a node and the value is a list of connected nodes.
14. Print the graph dictionary.
15. Count and print the total number of nodes and edges.

Phase 1 Reflection

After completing all six steps, write your answers to the following questions. Keep each answer to two or three sentences.

16. How many rows and columns does your dataset have? What does each row represent?
17. Are there missing values? If yes, which columns have them and how might this affect your analysis?
18. What is the target variable and how many unique classes does it have?
19. Describe the graph you built. What do the nodes and edges represent in the context of your domain?

Phase 2: Intelligent Agent and Search Algorithms

Objective

In this phase, you will design an intelligent agent that navigates your dataset using search algorithms. You will implement uninformed search (BFS, DFS, Depth Limited, Iterative Deepening, Uniform Cost Search), informed search (Best First, A*), and beyond classical search methods (Hill Climbing, Simulated Annealing, Local Beam Search). You will also implement Adversarial Search using Minimax and Alpha-Beta Pruning.

The goal of a search algorithm in the context of your dataset is to find the best path from a starting state (observed input features) to a goal state (the correct prediction or classification).

Step 1: Define the Agent and State Space

Before implementing any algorithm, you must clearly define the problem as a state space. Answer these questions in your code comments:

20. What is the initial state? For example, in Track A this might be a set of observed symptoms.
21. What is the goal state? For example, the correct disease diagnosis.
22. What are the possible actions? For example, checking whether the next symptom is present.
23. What is the cost of each action? Use the severity scores if available, otherwise treat all edges as cost 1.
24. What does a solution look like? For example, the path from the initial symptoms to the correct disease.

Create a Python class called `AIAgent` with the following methods:

6. `perceive(state)`: Takes the current state as input and returns a list of possible next states.
7. `act(action)`: Moves the agent to a new state.
8. `goal_test(state)`: Returns True if the current state is the goal state.
9. `get_cost(state1, state2)`: Returns the cost of moving from one state to another.

Step 2: Uninformed Search

Implement the following five uninformed search algorithms. Each must operate on the graph you built in Phase 1. For each algorithm, record the path found and the number of nodes explored.

Breadth First Search (BFS)

BFS explores all nodes at the current depth before going deeper. Use a queue (`collections.deque`). Start from a sample initial state in your dataset and find the goal state. Print the path and the total number of nodes visited.

Depth First Search (DFS)

DFS explores as far as possible along each branch before backtracking. Use a stack (Python list with `append` and `pop`). Run on the same initial and goal state as BFS. Compare the path length to BFS.

Depth Limited Search

Same as DFS but with a maximum depth limit. Set the limit to 3 first, then 5. Observe what happens when the goal is beyond the depth limit. Document whether a solution was found at each limit.

Iterative Deepening Search

This algorithm repeatedly calls Depth Limited Search with increasing depth limits starting from 0. Run it on your state space and observe at which depth level the goal is found. Compare the total nodes explored to BFS.

Uniform Cost Search

This expands the node with the lowest cumulative cost first. Use a priority queue (`heapq` module). Use the edge costs from your graph. This algorithm guarantees the lowest cost solution.

After completing all five, fill in the following comparison table in your notebook:

Algorithm	Path Found	Path Length	Nodes Explored	Optimal?
BFS				
DFS				
Depth Limited (limit=3)				
Depth Limited (limit=5)				
Iterative Deepening				
Uniform Cost Search				

Step 3: Informed Search

Informed search uses a heuristic function to guide the search towards the goal more efficiently. You must first define a heuristic function $h(\text{state})$ that estimates the cost from the current state to the goal.

A good heuristic for your dataset might be the number of matching features between the current state and the goal state, inverted so that more matches give a lower heuristic value.

Best First Search

This expands the node with the lowest heuristic value first. It does not consider path cost, only the estimated cost to the goal. Use a priority queue ordered by $h(\text{state})$. Run on the same problem and compare results to UCS.

A* Search

A* combines the actual path cost $g(\text{state})$ and the heuristic $h(\text{state})$ to compute $f(\text{state}) = g(\text{state}) + h(\text{state})$. Expand the node with the lowest f value. A* is both optimal and complete when the heuristic is admissible, meaning it never overestimates the true cost.

Verify your heuristic is admissible by checking that $h(\text{state})$ is always less than or equal to the true remaining cost for several sample states.

Step 4: Beyond Classical Search

These methods are used when the state space is too large for exhaustive search. They do not guarantee finding the optimal solution but are much faster.

Hill Climbing

Start at a random state. At each step, move to the neighboring state with the highest value according to an objective function. Stop when no neighbor is better. Run 10 times with different starting states and record how often you reach the global optimum versus a local optimum.

Simulated Annealing

Similar to Hill Climbing but occasionally accepts worse moves with a probability that decreases over time. This allows escape from local optima. Implement with a temperature parameter that starts at 100 and decreases by multiplying by 0.95 each step. Compare the final solution quality to Hill Climbing.

Local Beam Search

Maintain k candidate states simultaneously instead of one. At each step, generate all neighbors of all k states and keep the best k . Use $k = 3$ and $k = 5$. Compare the results to Hill Climbing in terms of solution quality and speed.

Step 5: Adversarial Search

Adversarial search is used when two agents compete. In the context of your dataset, you can model this as a two-player game where one player tries to reach the correct classification and the other tries to block it.

Minimax

Build a game tree of depth 4 from a starting state. The MAX player chooses the move that maximizes the evaluation function. The MIN player chooses the move that minimizes it. The evaluation function should reflect how close a state is to the correct label. Implement recursively and print the chosen move at each level.

Alpha-Beta Pruning

Implement Alpha-Beta Pruning on the same game tree. This optimization eliminates branches that cannot possibly affect the final decision. Count the number of nodes evaluated with and without pruning and record the difference.

Phase 2 Reflection

25. Which uninformed search algorithm explored the fewest nodes? Why?
26. Did your heuristic improve the search in A* compared to UCS? How do you measure improvement?
27. How often did Hill Climbing get stuck in a local optimum? Did Simulated Annealing help?
28. How many nodes did Alpha-Beta Pruning eliminate compared to plain Minimax?

Phase 3: Constraint Satisfaction

Objective

In this phase, you will model a problem from your dataset as a Constraint Satisfaction Problem (CSP). A CSP is defined by a set of variables, a domain of possible values for each variable, and a set of constraints that the solution must satisfy. You will implement constraint propagation, inference, backtracking, and local search.

Step 1: Define Your CSP

Translate a problem from your dataset into a formal CSP. Be specific and write down the following before coding:

29. Variables: What are the unknowns in your problem? For Track A, variables might be each symptom slot to be filled. For Track C, variables might be the study habits for a target grade.
30. Domains: What values can each variable take? List all possible values.
31. Constraints: What rules must the solution satisfy? Write at least five constraints as clear sentences before writing them as code.
32. Goal: What does a valid complete assignment look like?

Track	Example Variables	Example Constraints
Track A	Symptom slots for a diagnosis	If symptom A is present, symptom B must also be present to diagnose Disease X
Track B	Soil type, rainfall level, temperature range	Crop Y requires rainfall above 200mm and temperature below 30C
Track C	Study hours, attendance, parental support	A grade of A requires study hours above 4 per day and attendance above 80 percent
Track D	Work environment, treatment access, condition severity	Seeking treatment is required when severity is high and support is low

Step 2: Constraint Propagation and Inference

Constraint propagation reduces the domain of variables before searching. Implement the AC-3 algorithm (Arc Consistency 3) which removes values from variable domains that cannot be part of any valid solution.

33. Represent your CSP as a Python dictionary where each key is a variable and each value is a list of possible domain values.
34. Represent constraints as a list of pairs (variable_i, variable_j) where a constraint exists between them.
35. Implement the revise(csp, Xi, Xj) function that removes values from Xi's domain that have no consistent value in Xj's domain.
36. Implement AC-3 using a queue of all arcs. Process each arc and re-add neighbors when a domain changes.
37. Print the domain sizes before and after running AC-3.

Step 3: Backtracking Search

Backtracking search assigns values to variables one at a time and backtracks when a constraint is violated. Implement the following in order:

Basic Backtracking

Assign values to variables in a fixed order. When a constraint is violated, undo the last assignment and try the next value. Count the number of backtracks and print the final solution.

Backtracking with Forward Checking

After each assignment, check whether any remaining variable has an empty domain. If yes, backtrack immediately without exploring further. Compare the number of backtracks to basic backtracking.

Backtracking with MRV Heuristic

MRV stands for Minimum Remaining Values. Always assign next the variable with the fewest remaining domain values. This often reduces the number of backtracks significantly. Compare results to the two previous implementations.

Step 4: Local Search for CSP

Local search for CSPs starts with a complete assignment (which may violate constraints) and repairs it. Implement the Min-Conflicts algorithm:

38. Start with a random complete assignment of values to all variables.
39. Count the number of violated constraints in the current assignment.
40. Select a variable that is involved in at least one violated constraint.
41. Assign to that variable the value that minimizes the number of violated constraints.
42. Repeat until no constraints are violated or a maximum iteration count is reached.
43. Run for 100 iterations and record the number of violations at each step.

Phase 3 Reflection

44. How much did AC-3 reduce your domain sizes? Give exact numbers.
45. Which backtracking variant had the fewest backtracks? Why?
46. Did Min-Conflicts find a solution within 100 iterations? If not, how many violations remained?

Phase 4: Machine Learning

Objective

In this phase, you will apply machine learning techniques to your dataset. You will implement unsupervised learning to discover patterns without using labels, and then implement supervised learning using the Perceptron, the delta rule, and a multi-layer Artificial Neural Network with forward and backpropagation.

Step 1: Prepare Data for Machine Learning

Before running any machine learning algorithm, your data must be cleaned and formatted. Complete the following steps:

47. Handle missing values. Replace missing numerical values with the column mean. Replace missing categorical values with the most frequent value in that column.
48. Encode categorical variables. Convert text categories to numbers using label encoding. For example, Low becomes 0, Medium becomes 1, High becomes 2.
49. Normalize numerical features. Subtract the mean and divide by the standard deviation for each numerical column.
50. Split your dataset into training and testing sets. Use 80 percent for training and 20 percent for testing. Use a fixed random seed so results are reproducible.
51. Print the shapes of your training and testing sets to confirm the split.

Step 2: Unsupervised Learning

K-Means Clustering

K-Means groups data into k clusters by minimizing the distance between each point and its cluster center. Implement from scratch without using scikit-learn's KMeans class.

52. Choose k. Start with k equal to the number of unique classes in your target column.
53. Initialize k centroids by randomly selecting k rows from your dataset.
54. Assign each row to the nearest centroid using Euclidean distance.
55. Update each centroid by computing the mean of all rows assigned to it.
56. Repeat steps 3 and 4 until the centroids do not change between iterations or a maximum of 100 iterations is reached.
57. Print the final cluster assignments and centroids.

58. Compare the cluster assignments to the true labels. What percentage of rows in each cluster share the same true label?

K-Medoid Clustering

K-Medoid is similar to K-Means but uses actual data points as cluster centers (medoids) instead of computed means. This makes it more robust to outliers. Implement from scratch.

59. Initialize by randomly selecting k rows as medoids.
60. Assign each row to the nearest medoid.
61. For each cluster, find the point that minimizes the total distance to all other points in the cluster and set it as the new medoid.
62. Repeat until medoids do not change.
63. Compare the quality of K-Medoid clusters to K-Means clusters using the within-cluster sum of distances.

Step 3: Perceptron Training

A Perceptron is the simplest neural network. It is a single neuron that learns a linear decision boundary. Implement the Perceptron learning algorithm from scratch.

64. Initialize the weights as a vector of zeros with one weight per input feature, plus one bias weight.
65. For each training example, compute the weighted sum of inputs plus bias.
66. Apply the step activation function: output 1 if the sum is greater than 0, otherwise output 0.
67. Compute the error as the difference between the true label and the predicted output.
68. Update each weight using the Perceptron update rule: $\text{new_weight} = \text{old_weight} + \text{learning_rate} \times \text{error} \times \text{input}$.
69. Repeat for 50 epochs. Record the training accuracy after each epoch.
70. Plot the training accuracy over epochs.

Step 4: Delta Rule and Gradient Descent

The delta rule is used for training a single-layer network with a linear activation function. It minimizes the mean squared error using gradient descent.

71. Initialize weights as random small values.
72. For each training example, compute the linear output as the dot product of weights and inputs.
73. Compute the error as the true label minus the predicted output.

74. Update each weight using: $\text{new_weight} = \text{old_weight} + \text{learning_rate} \times \text{error} \times \text{input}$.
75. Implement batch gradient descent: compute the average weight update across all training examples before updating.
76. Run for 100 epochs. Record the mean squared error after each epoch.
77. Compare the convergence speed of the delta rule to the Perceptron.

Step 5: Multilayer Perceptron with Backpropagation

A multilayer perceptron (MLP) uses multiple layers of neurons to learn non-linear patterns. You will implement forward propagation and backpropagation from scratch.

Network Architecture

Use the following architecture. You may adjust the hidden layer size based on your dataset size.

Layer	Number of Neurons	Activation Function
Input Layer	Equal to number of features in your dataset	None
Hidden Layer 1	16 neurons	ReLU (max of 0 and input)
Hidden Layer 2	8 neurons	ReLU
Output Layer	Equal to number of classes	Softmax for multi-class, Sigmoid for binary

Forward Propagation

78. Multiply the input by the weights of the first hidden layer and add the bias.
79. Apply the ReLU activation function.
80. Multiply the result by the weights of the second hidden layer and add the bias.
81. Apply the ReLU activation function.
82. Multiply by the output layer weights and apply Softmax or Sigmoid.
83. The final output is the predicted probability for each class.

Backpropagation

84. Compute the loss using cross-entropy for multi-class or binary cross-entropy for two classes.
85. Compute the gradient of the loss with respect to the output layer weights.
86. Propagate the gradient backward through each hidden layer using the chain rule.
87. Update all weights using the computed gradients and the learning rate.
88. Repeat for 200 epochs. Record the loss and accuracy after every 10 epochs.

Activation Functions

Implement the following activation functions and their derivatives. You will need the derivatives during backpropagation.

Function	Formula	Use Case
ReLU	$f(x) = \max(0, x)$	Hidden layers
Sigmoid	$f(x) = 1$ divided by $(1 + e$ to the power of negative $x)$	Binary output layer
Softmax	$f(x_i) = e$ to the power of x_i divided by sum of all e to the power of x_j	Multi-class output layer
Tanh	$f(x) = (e^x - e^{-x})$ divided by $(e^x + e^{-x})$	Alternative hidden layer activation

Step 6: Compare All Models

After implementing all models, evaluate each one on your test set. Fill in the following table:

Model	Test Accuracy	Training Time (approx)	Notes
Perceptron			
Delta Rule (Linear)			
MLP with Backpropagation			
K-Means (Unsupervised, no labels)			Measure cluster purity instead

Phase 5 Reflection

89. Did K-Means clusters align with the true labels? What percentage of each cluster was a single class?
90. How many epochs did the Perceptron need to converge? Did it converge at all?
91. How much better was the MLP compared to the Perceptron? What does this tell you about the value of hidden layers?
92. Which activation function gave the best results in your hidden layers and why?

Phase 6: Research Report and Project Evaluation

Objective

In this final phase, you will compile your findings from all previous phases into a structured research report. You will compare your results to those of classmates on other tracks to answer the central research question of this project.

Step 1: Write Your Research Report

Your report must follow the structure below. Each section has a required minimum length. Write in clear, simple academic language. Every claim must be supported by data from your experiments.

Section	Content Required	Min Length
Abstract	A summary of what you did, what you found, and why it matters.	150 words
1. Introduction	Background on your domain. Why is AI useful for this problem? What is your research question?	300 words
2. Dataset Description	Describe your dataset in detail. Include rows, columns, classes, and any preprocessing steps.	200 words
3. Search Algorithms	Report results from Phase 2. Include the comparison table. Discuss which algorithm worked best and why.	400 words
4. Constraint Satisfaction	Report results from Phase 3. How many backtracks? Did AC-3 help? Did Min-Conflicts converge?	300 words
5. Logic and Reasoning	Report results from Phase 4. How accurate were your rules? What did forward chaining derive?	300 words
6. Machine Learning	Report results from Phase 5. Include the accuracy table. Which model performed best?	400 words
7. Cross-Track Comparison	Compare your results to at least two other tracks. What was similar? What was different?	300 words
8. Conclusion	What did you learn? What would you do differently? What future work could extend this project?	200 words
References	Cite your dataset source, any textbooks used, and any online resources.	N/A

Step 2: Cross-Track Comparison

This step requires collaboration with your classmates. Share only your results tables, not your code or dataset. Gather results from at least one student on each of the other three tracks. Then complete the following analysis:

93. For BFS and A* search: Did the number of nodes explored differ significantly across tracks? What might explain this?
94. For the Perceptron: Which track achieved the highest accuracy? Is the dataset size a factor?
95. For MLP: Which track had the highest test accuracy? Is the problem difficulty related to the number of classes?
96. For K-Means: Which track had the best cluster purity? What property of the data affects this?

Fill in the cross-track summary table with results gathered from your classmates:

Metric	Track A (Medical)	Track B (Agriculture)	Track C (Education)	Track D (Mental Health)
BFS Nodes Explored				
A* Nodes Explored				
Perceptron Accuracy				
MLP Test Accuracy				
K-Means Cluster Purity				

Step 3: Final Code Submission

Your final code submission must be organized as follows. Submit as a single ZIP file named using the format: StudentID_TrackLetter_AL2002.zip

Folder or File	Contents
phase1/	All Python files and Jupyter notebooks for Phase 1
phase2/	All search algorithm implementations
phase3/	All CSP implementations
phase4/	All logic and reasoning implementations
phase5/	All machine learning implementations
data/	Your dataset CSV files

report.pdf	Your research report exported as PDF
README.txt	A brief description of how to run each phase

Step 4: Viva Preparation

During the project evaluation session, you will be asked questions from the following areas. Prepare clear, concise answers. You may be asked to run your code live and explain the output.

Area	Example Questions
Data Understanding	What does each row in your dataset represent? How did you handle missing values?
Search Algorithms	Why did A* explore fewer nodes than BFS? What makes your heuristic admissible?
CSP	What were your variables and constraints? How many backtracks did your solution require?
Logic	Write a propositional logic rule derived from your data. Convert it to CNF.
Machine Learning	Explain backpropagation step by step. What does the learning rate control?
Research	What was your main finding? How did your results compare to other tracks?

Appendix: Quick Reference

A. Course Learning Outcomes Mapping

CLO	Description	Phases Covered
CLO 1	Python, intelligent agents, uninformed and informed search	Phase 1 and Phase 2
CLO 2	Constraint satisfaction using inference, backtracking, local search	Phase 3
CLO 3	Knowledge, reasoning, and planning using propositional and first-order logic	Phase 4
CLO 4	Supervised and unsupervised learning including Perceptron and ANN	Phase 5
CLO 5	Responsible professional behavior in lab and project	All Phases

B. Grading Rubric

Component	Weight	Criteria
Phase Completion (Phases 1-5)	40%	All steps completed, code runs without errors, reflection questions answered
Code Quality	15%	Code is readable, commented, and organized into functions and classes
Research Report	25%	All sections present, claims supported by data, cross-track comparison included
Viva Performance	15%	Clear explanations, able to run code live, able to answer follow-up questions
Professionalism	5%	Submission organized correctly, deadlines met, team behavior if applicable

C. Recommended Resources

10. Textbook: Artificial Intelligence - A Modern Approach by Russell and Norvig (any edition).
11. Python Documentation: docs.python.org
12. Pandas Documentation: pandas.pydata.org
13. NumPy Documentation: numpy.org
14. Kaggle Datasets: kaggle.com/datasets
15. For neural network visualization: playground.tensorflow.org

D. Academic Integrity Policy

Each student works on a different dataset Track. Code must be written independently. Sharing code between students, copying from online repositories, or using AI tools to write your implementation without understanding it are all considered violations of academic integrity.

You are expected to understand every line of code you submit. During the viva, you will be asked to explain specific sections of your code. If you cannot explain your code, marks will be deducted regardless of whether the code runs correctly.

You may discuss concepts and compare results with classmates. Sharing results tables for the cross-track comparison is required and encouraged. Sharing code is not permitted.

End of Lab Manual - AL2002 Artificial Intelligence Lab